SIMULADOR DE SISTEMA OPERATIVO BÁSICO EN CONSOLA (MINI-SO)

Documento técnico final

Integrantes
Deyver Valencia, Leiver Barrios, Jesús

Asignatura
Sistemas Operativos y Compiladores

Docente
Ing. Adriana Paredes Salinas

Programa
Ingeniería de Software

Institución
Universidad Cooperativa de Colombia

Año
2026

Introducción

El Mini-SO es un simulador didáctico de un sistema operativo básico que se maneja desde la consola. Está escrito en Python 3, en un único archivo (simulador.py, 223 líneas) y sin librerías externas. Integra cinco componentes que trabajan sobre un estado compartido: un intérprete de comandos, la gestión de procesos (PCB), una CPU con planificación Round Robin, una memoria RAM de 16 casillas con compactación y un sistema de archivos virtual (VFS).

Este documento describe la versión final del proyecto. Respecto a la Entrega 2 (versión 0.2, con FIFO), se completó todo lo que el cronograma dejaba pendiente: planificación Round Robin con quantum configurable, compactación de memoria, comando kill, ampliación del VFS (borrar y listar con tamaño), pruebas ampliadas de 6 a 12 y diagramas actualizados.

Objetivo y alcance

Objetivo general: simular, de forma controlada y visible, cómo un sistema operativo administra procesos, CPU, memoria y archivos, de modo que el usuario pueda observar cada decisión (reserva de RAM, cambio de proceso, liberación, fragmentación) mediante comandos.

Alcance: simulación en una sola consola, sin hilos ni concurrencia real. El tiempo avanza solo cuando el usuario escribe step (un tick por comando). La RAM es una lista de 16 casillas y el disco virtual es un diccionario en memoria; por eso los archivos no persisten al cerrar el programa.

Requerimientos y cumplimiento

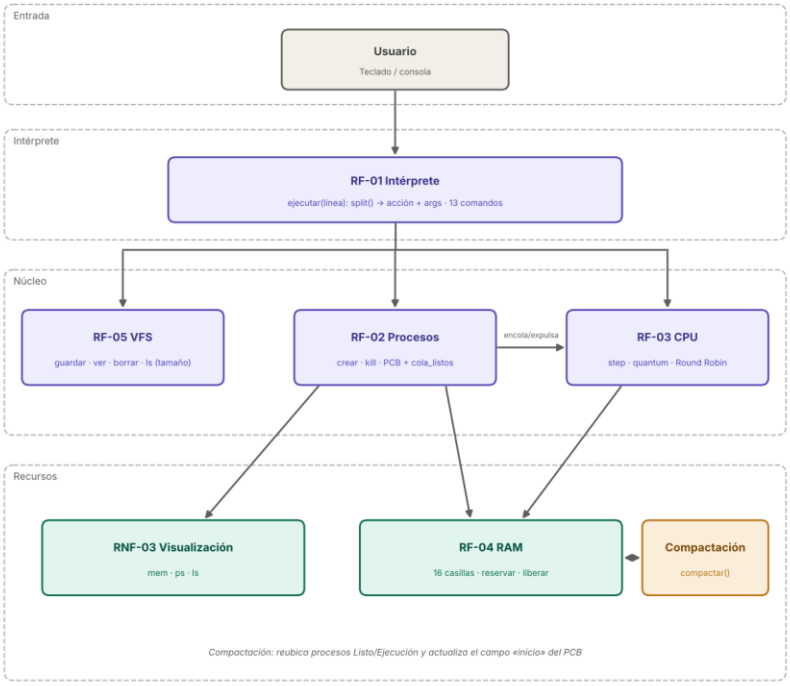
Código	Módulo	Implementación en simulador.py	Estado
RF-01	Intérprete por consola	ejecutar(linea) separa con split() y despacha los 13 comandos. Valida argumentos con try/except y avisa de comandos desconocidos.	Cumplido
RF-02	Gestión de procesos	crear() asigna PID incremental y registra el proceso en el diccionario pcb (nombre, tam, inicio, restante, estado). kill() elimina un proceso activo y libera su RAM.	Cumplido
RF-03	Control de CPU	step() avanza un tick con planificación Round Robin: si el proceso agota el quantum y hay otros listos, vuelve al final de cola_listos. quantum [n] consulta o cambia el valor.	Cumplido
RF-04	Gestión de RAM	16 casillas. reservar() usa primer ajuste en bloques continuos, liberar() borra los bloques del proceso y compactar() reubica los procesos activos al inicio para eliminar la fragmentación.	Cumplido
RF-05	Sistema de archivos (VFS)	Diccionario vfs {nombre: texto} con guardar, ver, borrar y ls (que muestra el tamaño de cada archivo y el total).	Cumplido

Requerimientos no funcionales:

Código	Requerimiento	Verificación	Estado
RNF-01	Un solo archivo de código, de 100 a 150 líneas	Un solo archivo, pero con 223 líneas. Las cuatro funciones nuevas pedidas para la entrega final (Round Robin, compactación, kill, ampliación del VFS) más sus comentarios no caben en 150 líneas sin sacrificar legibilidad. Se mantiene el archivo único y se documenta la desviación (ver sección 11).	Cumplido parcialmente (un archivo; excede el rango de líneas)
RNF-02	Solo Python 3, sin instalar librerías	simulador.py y pruebas.py solo usan la biblioteca estándar (io, contextlib).	Cumplido
RNF-03	Visualización directa del estado	mem muestra la RAM como una barra de 16 casillas; ps muestra la tabla de procesos, el proceso en CPU, el quantum y la cola de listos; ls muestra los archivos con su tamaño.	Cumplido

Arquitectura

El sistema se organiza en cuatro capas: entrada (el usuario), intérprete, núcleo (procesos, CPU y VFS) y recursos (RAM y visualización). El intérprete es la única puerta de entrada: recibe una línea, la separa en acción y argumentos y llama al módulo correspondiente.



Cambios respecto a la Entrega 2: la CPU ahora expulsa procesos por quantum y los devuelve a la cola; Procesos agrega kill, que libera RAM; la RAM agrega compactar(); y el VFS agrega borrar y listado con tamaño.

Componentes

Los componentes se implementaron como funciones que comparten el estado global del sistema. No hay clases: se priorizó que el código fuera simple de leer y explicar.

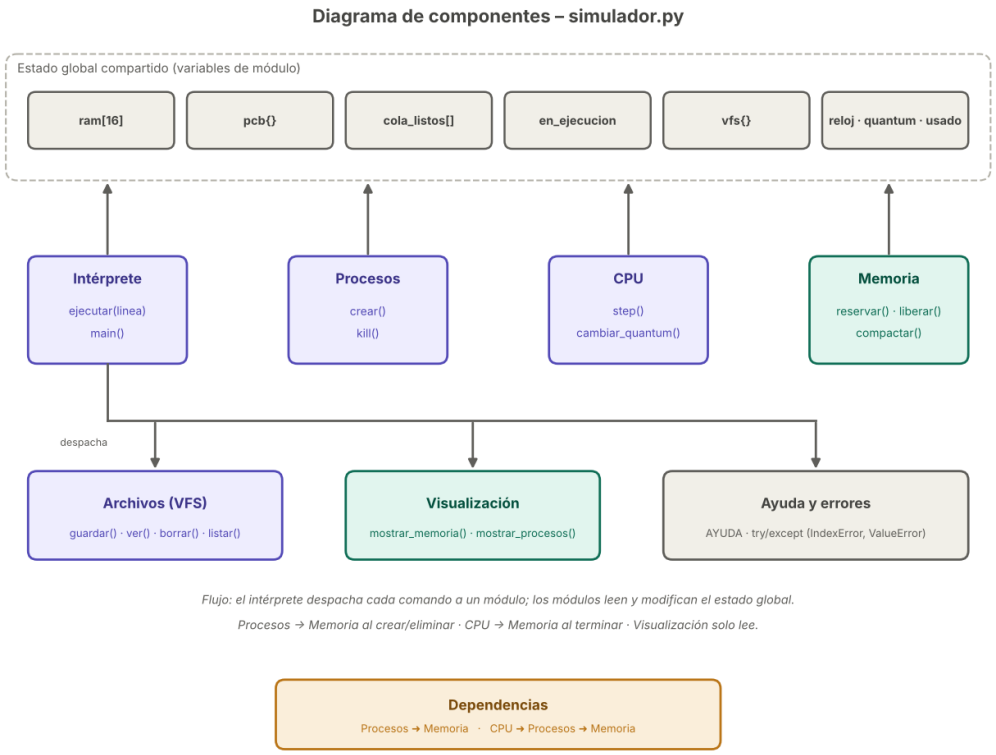


Figura 2. Diagrama de componentes: funciones de cada módulo y estado global que comparten.

Componente	Funciones	Responsabilidad
Intérprete	`ejecutar(), main()`	Leer comandos, validar argumentos y despachar. ejecutar() está separada de input() para poder probarla automáticamente.
Procesos	`crear(), kill()`	Crear procesos (reserva RAM antes de registrarlos) y eliminarlos.
CPU	`step(), cambiar_quantum()`	Avanzar el reloj, descontar ticks, terminar o expulsar procesos.
Memoria	`reservar(), liberar(), compactar()`	Administrar las 16 casillas de RAM.
Archivos	`guardar(), ver(), borrar(), listar()`	Administrar el VFS.
Visualización	`mostrar_memoria(), mostrar_procesos()`	Mostrar el estado del sistema de forma directa (RNF-03).

Estructuras de datos

Variable	Tipo	Contenido
`ram`	lista de 16 elementos	Cada casilla guarda el PID que la ocupa o None si está libre.
`pcb`	dict {pid: dict}	Tabla de procesos. Cada entrada tiene nombre, tam, inicio, restante y estado.
`cola_listos`	lista de PIDs	Procesos en estado Listo, en orden de llegada (cola del Round Robin).
`en_ejecucion`	PID o None	Proceso que tiene la CPU.
`vfs`	dict {nombre: texto}	Disco virtual.
`reloj, quantum, usado`	enteros	Tick global; ticks máximos seguidos por proceso (2 por defecto); ticks que lleva el proceso actual en CPU.

Algoritmos principales

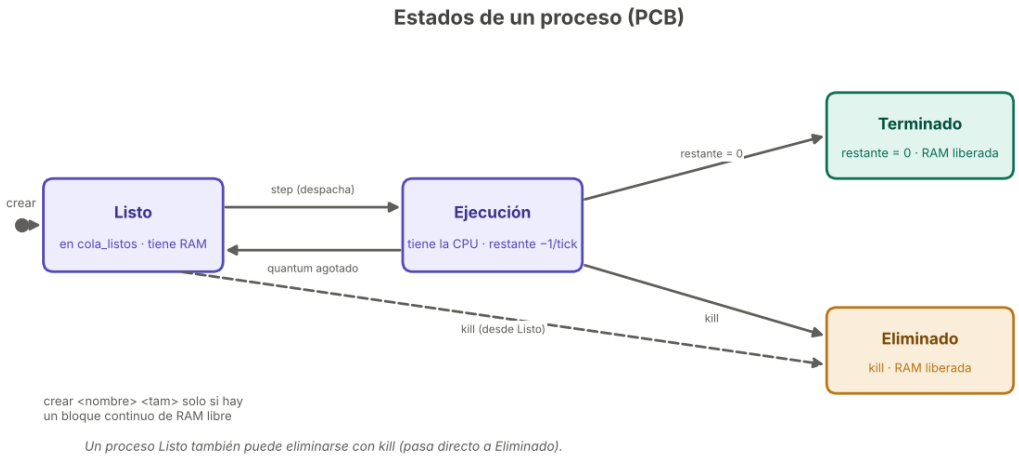


Figura 3. Diagrama de estados del PCB.

Un proceso nace Listo (solo si hay RAM). step() lo pasa a Ejecución. Desde ahí puede volver a Listo (quantum agotado), pasar a Terminado (sin ticks) o a Eliminado (kill). Terminado y Eliminado liberan la RAM y el proceso queda en la tabla como historial.

Primer ajuste (first-fit)

reservar() recorre la RAM una sola vez contando casillas libres consecutivas; en cuanto el contador alcanza el tamaño pedido, asigna ese bloque. Si llega al final sin lograrlo, devuelve -1 y crear() informa el error aunque haya suficientes casillas libres sueltas (fragmentación externa).

Round Robin

Cada step() ejecuta un tick. Si no hay proceso en CPU, toma el primero de cola_listos y reinicia el contador usado. Después descuenta un tick y evalúa:

- Si restante llega a 0: el proceso pasa a Terminado y libera su RAM.
- Si usado >= quantum y hay otros procesos listos: el proceso pasa a Listo y se coloca al final de cola_listos (expulsión).
- Si no hay otros procesos esperando, el proceso sigue en CPU aunque haya agotado el quantum, porque expulsarlo no cambiaría nada.

```
elif usado >= quantum and cola_listos:      # quantum agotado y hay otros esperando
    p["estado"] = "Listo"
    cola_listos.append(en_ejecucion)
    en_ejecucion = None
```

Compactación

compactar() ordena los procesos activos (Listo o Ejecución) por su casilla inicial y los desplaza uno a uno hacia el comienzo de la RAM, actualizando el campo inicio del PCB. Al final, todas las casillas libres quedan juntas al final. Como cada proceso solo se mueve hacia posiciones menores o iguales a la actual, nunca pisa a otro. Los procesos terminados o eliminados no se tocan: ya no tienen RAM.

Decisiones técnicas

Decisión	Justificación
Primer ajuste en RAM	Es el algoritmo más simple de explicar y cumple RF-04 con pocas líneas.
Round Robin con quantum configurable (2 por defecto)	Cumple el planteamiento de la entrega final. El control manual por ticks se mantiene: un step = un tick.
Expulsar solo si hay otros procesos listos	Evita mensajes de expulsión sin efecto cuando hay un único proceso.
Estado Eliminado separado de Terminado	En ps se distingue si el proceso acabó por sí mismo o por kill.
Compactación manual (comando compactar)	Deja que el usuario vea primero la fragmentación y luego su solución, que es lo que se quiere mostrar en la sustentación.
Estado en variables globales de módulo	Evita clases y mantiene el código legible. Sin hilos, como se acordó en la delimitación.
ejecutar(linea) separada de input()	Permite probar el intérprete sin escribir en consola (pruebas.py).
Procesos terminados permanecen en pcb	Se conserva el historial para ps; solo se libera su RAM. ps muestra «-» en vez de un rango de RAM.
Argumentos inválidos (IndexError, ValueError)	La consola no se cae. Tamaño, ticks y quantum menores que 1 también se rechazan.

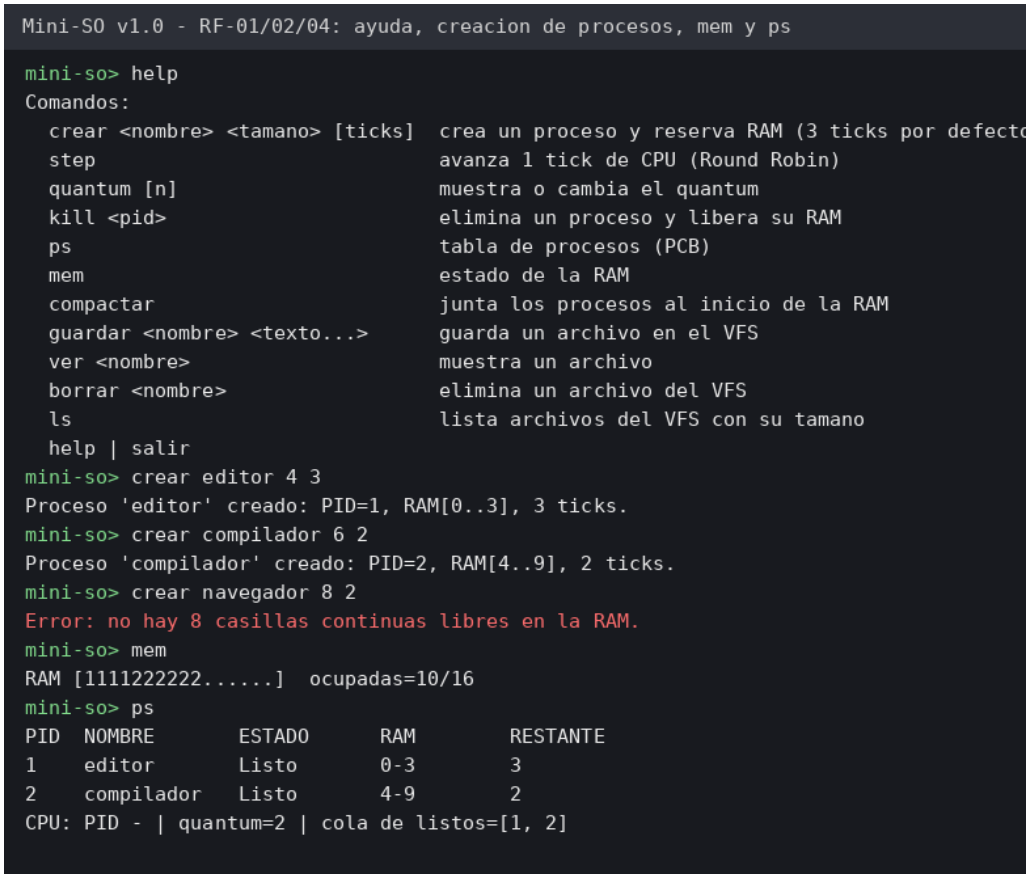
Pruebas funcionales

Las pruebas están automatizadas en pruebas.py (python pruebas.py). Cada prueba reinicia el estado del sistema, ejecuta comandos reales mediante ejecutar() y comprueba el resultado con assert. Resultado: 12 de 12 pruebas aprobadas.

Prueba	Req.	Caso y resultado esperado	Resultado
t01	RF-01	Comando inexistente y argumentos faltantes muestran un error sin cerrar el programa.	Aprobada
t02	RF-02	Un proceso nuevo queda Listo, pasa a Ejecución con step y a Terminado al agotar sus ticks.	Aprobada
t03	RF-03	step sin procesos informa CPU ociosa; con un proceso descuenta exactamente 1 tick.	Aprobada
t04	RF-04	No reserva si no hay bloque continuo; libera al terminar y reutiliza el espacio desde la casilla 0.	Aprobada
t05	RF-05	guardar y ver devuelven el mismo texto; ver de un archivo inexistente informa el error.	Aprobada
t06	Integración	Tres procesos se crean, ejecutan y terminan; al final la RAM queda vacía.	Aprobada
t07	RF-03	Round Robin: con quantum 2, un proceso de 3 ticks es expulsado tras 2 ticks y la cola queda [2, 1]; al final todos terminan en 6 ticks.	Aprobada
t08	RF-03	quantum muestra y cambia el valor; rechaza 0 y texto; con quantum 5 el proceso no es expulsado antes de tiempo.	Aprobada
t09	RF-02	kill elimina un proceso en CPU y otro en cola, libera su RAM y rechaza PIDs inexistentes, ya eliminados o ausentes.	Aprobada
t10	RF-04	Con 4+4 casillas libres separadas no cabe un proceso de 6; tras kill y compactar el proceso se reubica en 0-3 y cabe uno de 12.	Aprobada
t11	RF-05	ls vacío, ls con tamaños y total, borrar existente e inexistente, ver tras borrar.	Aprobada
t12	Integración	Escenario completo con Round Robin, kill, compactar y VFS: todos los procesos terminan, la RAM queda vacía y el archivo sigue guardado.	Aprobada

Evidencias de funcionamiento

Las capturas corresponden a sesiones reales de la versión final del simulador (la salida se obtuvo ejecutando cada comando sobre simulador.py).



Captura 1. Ayuda, creación de procesos, error por falta de RAM, y comandos mem y ps.

```
Mini-SO v1.0 - RF-03: Round Robin y comando quantum

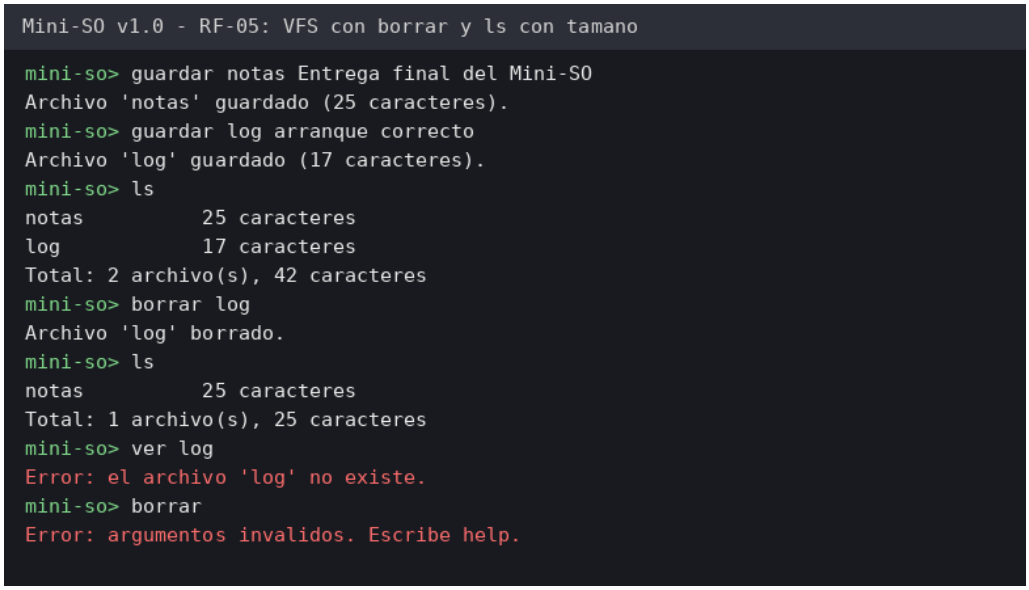
mini-so> crear editor 4 3
Proceso 'editor' creado: PID=1, RAM[0..3], 3 ticks.
mini-so> crear compilador 6 3
Proceso 'compilador' creado: PID=2, RAM[4..9], 3 ticks.
mini-so> crear navegador 4 2
Proceso 'navegador' creado: PID=3, RAM[10..13], 2 ticks.
mini-so> step
[tick 1] PID 1 (editor) ejecuta, le quedan 2 ticks.
mini-so> step
[tick 2] PID 1 (editor) ejecuta, le quedan 1 ticks.
  -> quantum agotado: PID 1 vuelve a la cola de listos.
mini-so> step
[tick 3] PID 2 (compilador) ejecuta, le quedan 2 ticks.
mini-so> ps
PID  NOMBRE       ESTADO      RAM        RESTANTE
1    editor       Listo       0-3        1
2    compilador   Ejecucion   4-9        2
3    navegador    Listo       10-13      2
CPU: PID 2 | quantum=2 | cola de listos=[3, 1]
mini-so> quantum 3
Quantum = 3 ticks.
mini-so> quantum 0
Error: argumentos invalidos. Escribe help.
```

Captura 2. Round Robin: expulsión por quantum, cambio de proceso y comando quantum.

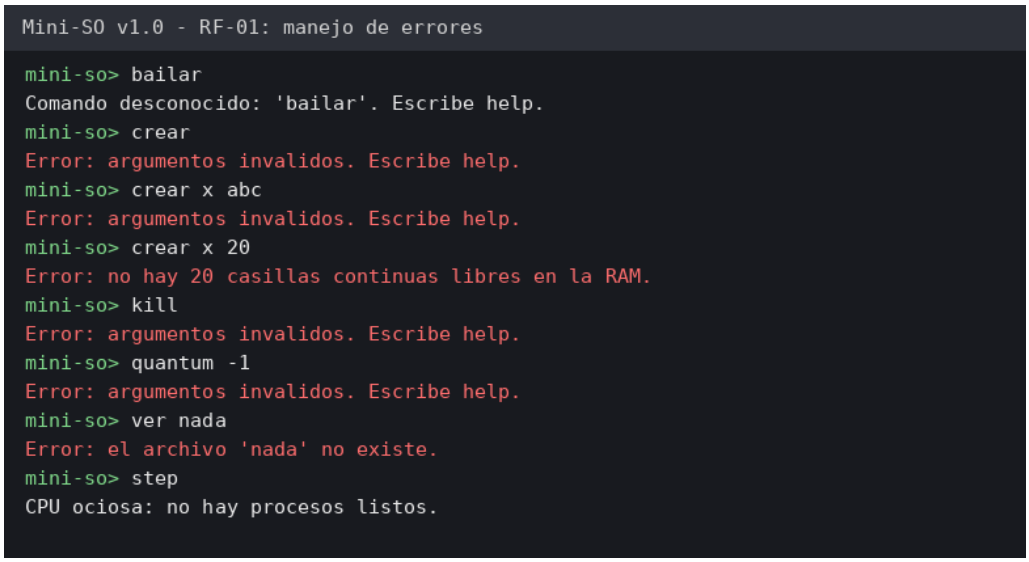
```
Mini-SO v1.0 - RF-02/04: kill, fragmentacion y compactacion

mini-so> crear a 4 5
Proceso 'a' creado: PID=1, RAM[0..3], 5 ticks.
mini-so> crear b 6 5
Proceso 'b' creado: PID=2, RAM[4..9], 5 ticks.
mini-so> crear c 4 5
Proceso 'c' creado: PID=3, RAM[10..13], 5 ticks.
mini-so> kill 2
Proceso PID 2 (b) eliminado; memoria liberada.
mini-so> mem
RAM [1111......3333..]  ocupadas=8/16
mini-so> crear grande 8 2
Error: no hay 8 casillas continuas libres en la RAM.
mini-so> compactar
Memoria compactada: 8 casillas libres continuas (RAM[8..15]).
mini-so> mem
RAM [11113333........]  ocupadas=8/16
mini-so> crear grande 8 2
Proceso 'grande' creado: PID=4, RAM[8..15], 2 ticks.
mini-so> kill 99
Error: no hay un proceso activo con PID 99.
```

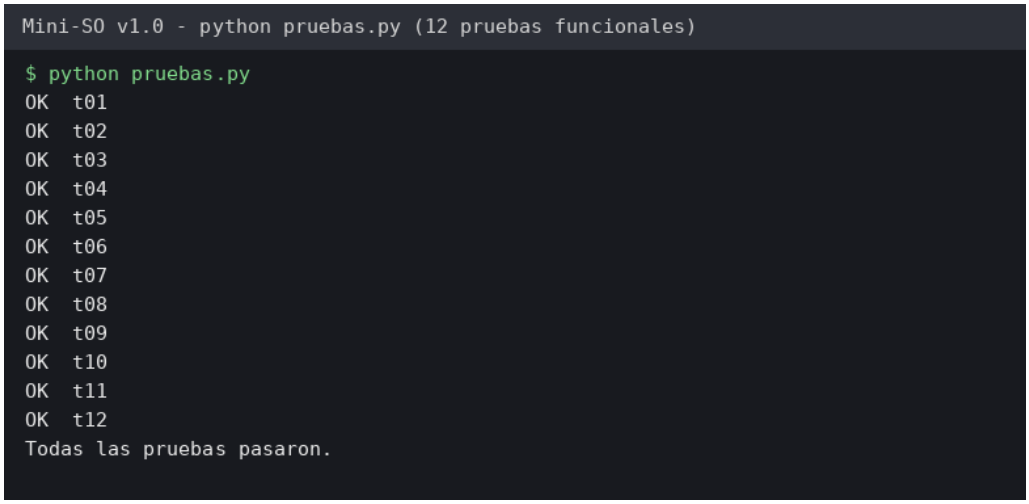
Captura 3. Kill, fragmentación (4+4 libres no alcanzan para 8) y solución con compactar.



Captura 4. Sistema de archivos virtual: guardar, ls con tamaño, borrar y errores.



Captura 5. Manejo de errores del intérprete.



Captura 6. Ejecución de las 12 pruebas funcionales automáticas.

Repositorio y organización del código

```
simulador.py          código fuente del Mini-SO (un solo archivo)
pruebas.py            pruebas funcionales automáticas (t01..t12)
demo.txt              sesión de ejemplo (entrada)
demo_salida.txt       salida real de la sesión de ejemplo
README.md             instrucciones de uso y lista de comandos
docs/                 informe técnico, manual de usuario, presentación,
                      diagramas, capturas y salida de las pruebas
```

Ejecución: python simulador.py (requiere Python 3). Pruebas: python pruebas.py. Enlace del repositorio:

Cumplimiento del cronograma

Semana	Actividad planificada	Estado
Semana 11	Planificación Round Robin con quantum configurable (comando quantum)	Implementado
Semana 12	Compactación de memoria y comando para eliminar procesos (kill)	Implementado
Semana 13	Ampliar el VFS (borrar y listar con tamaño) y ampliar pruebas funcionales	Implementado (12 pruebas)
Semana 14	Verificación de RNF, revisión de código y actualización de diagramas	Realizado (ver sección 3 y 11; diagramas actualizados)
Semana 15	Informe técnico final	Informe, manual y presentación incluidos.
Semana 16	Entrega final y sustentación	Pendiente